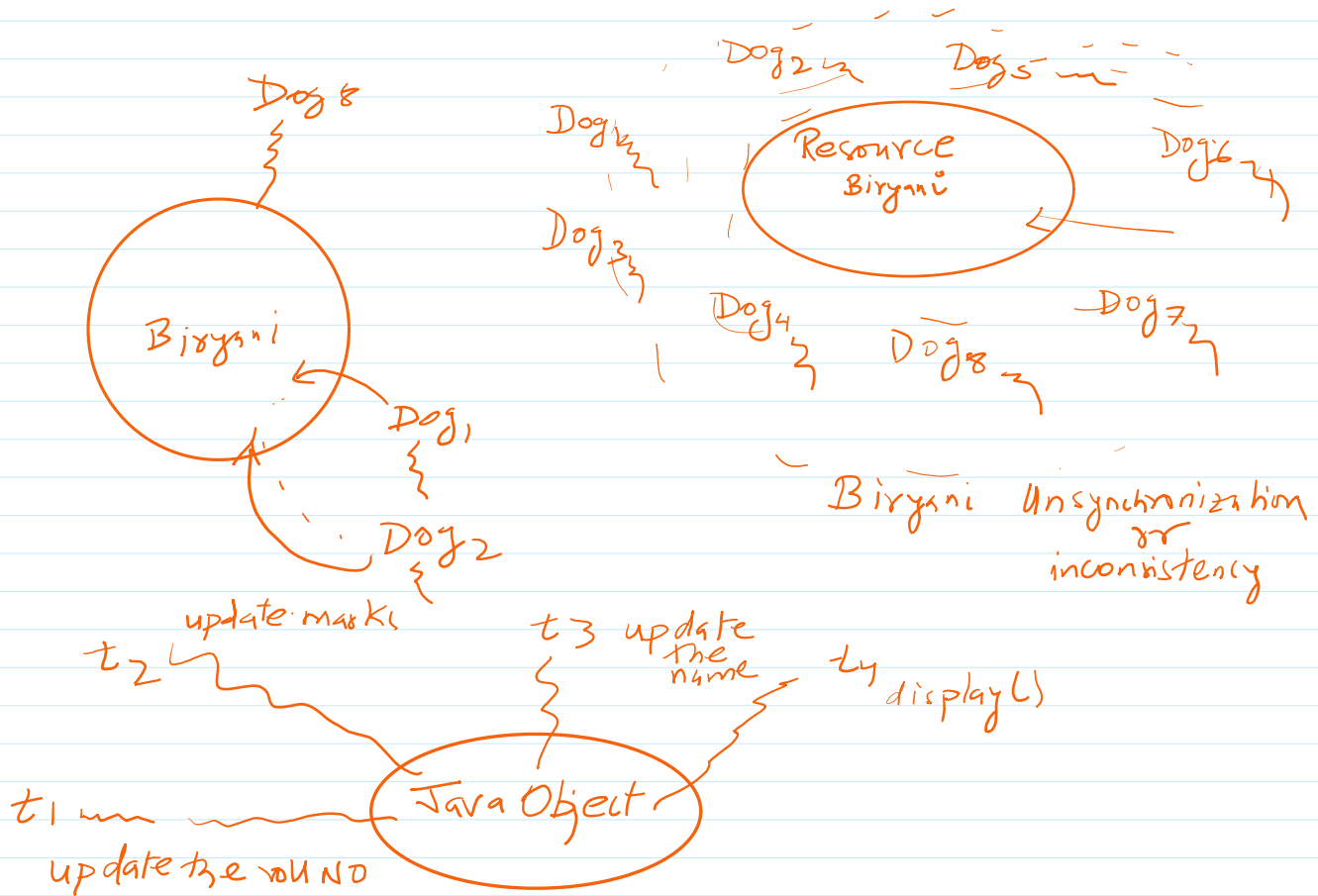


# Synchronization

16 March 2026 18:26



## Data inconsistency Problem

'synchronized' is a modifier applicable only for method and block a part for classes and variables.

If multiple threads are trying to operate simultaneously on the same Java object there may be chance of data inconsistency problem.

to Java Object, to overcome this problem we should go for "synchronized" keyword. If your method block are declared as "synchronized" then at a time only one thread is allowed to execute. That method is locked on the given object, so that data inconsistency problem will be resolved.

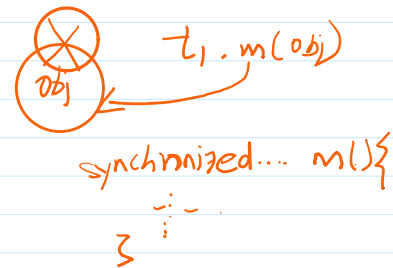
The main advantage of "synchronized" keyword is we can resolve data inconsistency problem. But the main disadvantage of "synchronized"

inconsistency problem. But the main disadvantage of "synchronized" keyword is it increases waiting time for other threads and creates performance problem. Hence, if there is no specific requirement then it is not recommended to use "synchronized" keyword.

Internally synchronization concept is implemented using lock.

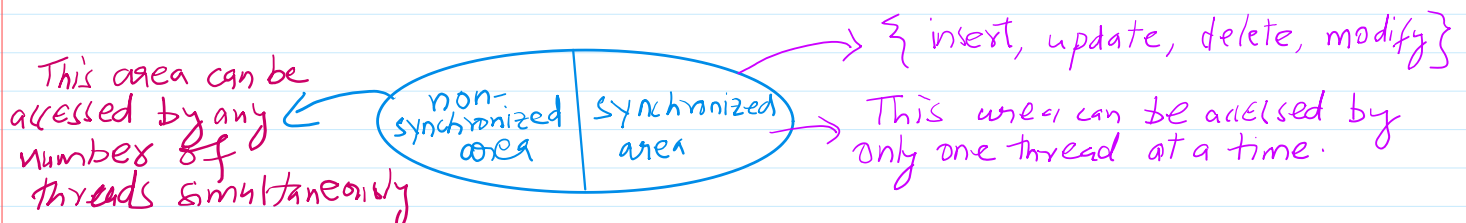
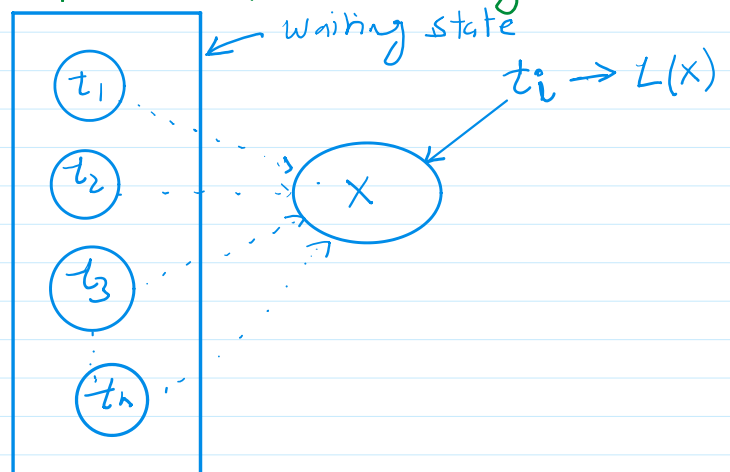
Every object in Java has a unique lock, whenever we are using 'synchronized' keyword, then only lock concept come into the picture. If thread want to

execute synchronized method on the given object, first it has to get the lock of that object, once thread got the lock, then it is allowed to execute any synchronized method on that object.



Once method execution completes automatically Thread releases the lock. Acquiring and releasing lock internally taken care by JVM and programmer is not responsible for this activity.

```
class X {
    sync ... m1() {}
    sync ... m2() {}
    sync ... m3() {}
}
```



While a thread executing synchronized method on the given object,

While a thread executing synchronized method on the given object, the remaining thread are not allowed to execute any synchronized method simultaneously on the same object. But remaining threads are allowed to execute non-synchronized method simultaneously.

```
class X {
```

```
    synchronized Area {
```

whenever we are performing update operation  
(add/remove/delete/replace(modify))  
i.e., where the state of the object is changing

```
    }
```

```
    non-synchronized Area {
```

where the object state won't be changing  
like read operation.

```
    }
```

```
}
```

Example :

```
class ReservationSystem {
```

```
    (Non-synchronized) checkAvailability() {
```

just read operation

```
    }
```

```
    (Synchronized) book_ticket() {
```

update database

```
    }
```

```
}
```

```
package ThreadSynchronization;
public class Display {
    //non-synchronized
    public void wish(String name){
```

```

        for (int i = 0; i < 10; i++) {
            System.out.println("Good morning");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println(name);
        }
    }
}

```

```

package ThreadSynchronization;
public class ThreadSync extends Thread{
    private Display d;
    private String name;
    public ThreadSync(Display d, String name){
        this.d = d;
        this.name = name;
    }
    @Override
    public void run() {
        d.wish(name);
    }
}

```

```

package ThreadSynchronization;
public class SynchronizedDemo {
    public static void main(String[] args) {
        Display d = new Display();
        ThreadSync ts1 = new ThreadSync(d, "Rohit");
        ThreadSync ts2 = new ThreadSync(d, "Virat");
        ts1.start();
        ts2.start();
    }
}

```

Output:

```

Good morning
Good morning
Rohit
Good morning
Virat
Good morning
Virat
Good morning
Rohit
Good morning
Virat
Good morning
Rohit
Good morning
Virat

```

|              |
|--------------|
| Rohit        |
| Good morning |
| Rohit        |
| Good morning |
| Virat        |
| Good morning |
| Virat        |
| Good morning |
| Rohit        |
| Good morning |
| Virat        |
| Good morning |
| Rohit        |
| Good morning |
| Virat        |
| Good morning |
| Rohit        |
| Good morning |
| Virat        |
| Good morning |
| Rohit        |
| Good morning |
| Virat        |
| Rohit        |

```
package ThreadSynchronization;
public class Display {
    //synchronized
    public synchronized void wish(String name){
        for (int i = 0; i < 10; i++) {
            System.out.println("Good morning");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println(name);
        }
    }
}
```

- Output is predictable when threads are working on same object.

```
package ThreadSynchronization;
public class SynchronizedDemo {
    public static void main(String[] args) {
        Display d = new Display();
        ThreadSync ts1 = new ThreadSync(d, "Rohit");
        ThreadSync ts2 = new ThreadSync(d, "Virat");
        ThreadSync ts3 = new ThreadSync(d, "Surya");
        ThreadSync ts4 = new ThreadSync(d, "Samsun");
        ts1.start();
    }
}
```

```
        ts2.start();  
        ts3.start();  
        ts4.start();  
    }  
}
```

Output:

Good morning  
Rohit  
Good morning  
Rohit  
Good morning  
Rohit  
Good morning  
Rohit  
Good morning  
Rohit  
Good morning  
Rohit  
Good morning  
Rohit  
Good morning  
Rohit  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Samsun  
Good morning  
Surya  
Good morning  
Surya  
Good morning  
Surya  
Good morning

Surya  
Good morning  
Surya  
Good morning  
Surya  
Good morning  
Surya  
Good morning  
Surya  
Good morning  
Surya  
Good morning  
Surya  
Good morning  
Virat  
Good morning  
Virat  
Good morning  
Virat  
Good morning  
Virat  
Good morning  
Virat  
Good morning  
Virat  
Good morning  
Virat  
Good morning  
Virat  
Good morning  
Virat  
Good morning  
Virat

Output is unpredictable, because threads are operating on different object. Even though wish() method is synchronized.